

5 Modelling and Aggregating Social Network Data

This chapter focuses on applying **ontology-based Knowledge Representation (KR) techniques** to handle and combine social network data. There are two main reasons for using semantic-based representations for social networks:

1. **Aggregating heterogeneous data:** It is crucial for combining social network information, especially when data comes from diverse sources that are under different controls, like various online communities or intelligence agencies. This is most beneficial when data is already in electronic format.
2. **Facilitating data exchange and reuse in academia:** Semantic representations can revolutionize the exchange and reuse of case study data in Social Network Analysis (SNA), similar to how it transformed bioinformatics and genetics. Currently, network data is often stored in formats not primarily designed for network analysis (e.g., Excel, SPSS) or in proprietary graph description languages that ignore the *semantics* (meaning) of the data, such as instance types or attribute meanings. This makes it difficult to independently verify results, reuse data for secondary analysis, or compare results across different studies. This lack of comparability is particularly problematic because single studies in network analysis typically focus on small communities, while the field aims to explain commonalities across all social networks.

The text first discusses the current state of network data representation and then motivates the need for semantics. It separately covers semantic-based representations of **social individuals** (Section 5.2) and **social relationships** (Section 5.3). Representing social individuals is relatively straightforward, but social relationships are much more challenging, making the development of an ontology for social relations a significant contribution.

While reasoning with social relations is still future work, the aggregation of social individuals can be well automated based on current representations and their formal semantics. This aggregation is a key technical component of an "end-to-end system" for collecting, aggregating, and presenting social network data.

5.1 State-of-the-art in network data representation

Social network data is commonly modeled as a **graph**, where **nodes** represent individuals and **edges** represent binary social relationships. Less commonly, higher-arity relationships (connecting multiple nodes) can be represented using hyper-edges. Additionally, social network studies use **attributes** for nodes and edges, which can be formalized as functions operating on them.

There are many **proprietary formats** for storing these graphs and attribute data in machine-processable electronic documents. Popular examples include formats used by network analysis packages like **Pajek** and **UCINET**. These are text-based and easy to edit, but they are incompatible with each other (though UCINET can read Pajek's `.net` format). Researchers in social sciences often start with **Microsoft Excel spreadsheets** and export them as **CSV (Comma Separated Values)**, a generic table format that requires additional constraints to be processed by graph packages. Visualization software like **GraphViz** also has its own proprietary formats, such as the `dot` format. A simple graph described in Pajek `.NET`, UCINET DL, and GraphML formats is shown in Figure 5.1 (refer to source for visual example).

GraphML is an advancement in terms of **interoperability** and **extensibility**. It originated from the information visualization community to promote a shared format. GraphML is based on **XML** with a schema defined in **XML Schema**, allowing generic XML tools to edit, store, query, and transform GraphML files. However, common to all these generic graph representations is their focus on the *graph structure*, which is the primary input for network analysis and visualization. Attribute data is typically stored separately in Excel, databases, or SPSS tables. While GraphML does allow for extensions to attach extra attributes to nodes, it doesn't provide a standard way to define the *types* of nodes or edges, nor does XML allow multiple types for an element.

Crucially, **none of these formats support the aggregation and reuse of electronic data**, which is a primary concern for the author. To illustrate the need for data aggregation, consider a scenario (Figure 5.2, refer to source for visual example) where a network study needs to reuse multiple data sources describing the same individuals and relationships (e.g., email archives and publication databases). Using multiple data sources often aims for **triangulation**, where different sources or methods verify the same conclusion. Data sources might also contain complementary information for **multi-layer studies**, which examine multiplex networks with various kinds of relationships (e.g., how different networks differ or how one relationship type affects another).

In both aggregation scenarios (triangulation and multi-layer studies), it's essential to **recognize matching instances** across different data sources and **merge their records** before analysis. The graph representations discussed earlier strip social network data of precisely these characteristics needed for aggregation and sharing, as they reduce individuals and relationships to mere nodes and edges, sufficient only for analyzing single networks.

What is needed for aggregation and reuse is a representation that can **capture and compare the identity of instances and relationships**. Maintaining identity is also vital for preserving datasets for reuse and secondary analysis. This problem requires a very different representation from graph-based formats. The proposed solution is a **rich, semantic-based representation** of social network primary objects: social individuals and their relationships. A semantic-based representation allows leveraging **ontology languages and tools** for data aggregation by incorporating **domain-specific knowledge about identity** (what makes two instances the same). This approach also enables enriching datasets with domain knowledge, such as inferring that if two people email each other, they know each other, or that one relationship type implies or refutes another.

The two key problems in aggregating social network data are the **identification and disambiguation of social individuals** and the **aggregation of information about social relationships**. These are treated separately.

5.2 Ontological representation of social individuals

The **Friend-of-a-Friend (FOAF) ontology** is introduced as an **OWL-based format** for representing personal information and an individual's social network. FOAF surpasses traditional graph description languages in **expressivity** by using the powerful OWL vocabulary. More importantly, it leverages the **extensibility of the RDF/OWL representation framework** to enhance the basic ontology with domain-specific knowledge about identity.

FOAF deployment studies show that most FOAF profiles on the Web are **auto-generated** by community sites like LiveJournal, Opera Communities, and Tribe.net. Estimating the total number of FOAF profiles is difficult due to their distributed nature, but even manually maintained profiles are in the tens of thousands.

FOAF originated as an experiment with **Semantic Web technology**, aiming to provide a machine-processable format for the personal information found on early homepages. It includes vocabulary for common homepage attributes like name, email, projects, interests, and links to work/school homepages. FOAF profiles can also describe friends using the same vocabulary and can be **linked together to form networks of web-based profiles**.

FOAF gained significant interest around 2003 with the rise of centralized **Social Networking Services (SNS)** like Friendster, Orkut, and LinkedIn. However, users later discovered several drawbacks to these centralized services:

1. **Control of information:** Data is controlled by the database owner, who has an interest in keeping information tied to their site and protects it

legally and technically. Profiles typically cannot be exported in machine-processable formats, preventing data transfer between systems (leading to separate data maintenance across services).

2. **User control:** Centralized systems often don't allow users to control their information on their own terms, leading to unintended uses of their data.

These problems are addressed by **Semantic Web technology**. Unlike centralized SNS, FOAF profiles are **created and controlled by individual users** and shared in a **distributed fashion**. They are usually posted on personal websites and linked from homepages using HTML META tags. The distributed nature of FOAF networks requires a mechanism to link profiles for discovery, which FOAF achieves using the `rdfs:seeAlso` mechanism. **Scutters (RDF crawlers)** can traverse these links to discover related profiles. However, FOAF profiles generated by online communities (like LiveJournal) often only link to members within the *same* website, creating "black holes" for crawling, as there are no outbound links. Search engines like Google can also discover FOAF profiles linked via meta tags or regular HTML anchor tags.

The distributed nature of FOAF data necessitated addressing **identification and aggregation** issues early on. Centralized solutions, such as a registry for individual URIs, were deemed infeasible. The solution is to identify persons using their **characteristic properties** like email addresses or homepages. When describing a friend, one might create a **blank node** with enough detail for unique identification. Blank nodes in RDF represent unidentified resources, similar to an existential quantifier in logic (e.g., "There exists a Person named Peter... who knows a person named Dirk...").

With the introduction of **OWL**, it became possible to describe these identifying properties of the `foaf:Person` class as **Inverse-Functional Properties (IFP)**. IFPs are properties whose value uniquely identifies a single object. This means that if two resources share the same value for an IFP, they *must* denote the same object. For example, `foaf:mbox` (email address) is an IFP because each email address typically belongs to only one person (excluding generic addresses like `support@microsoft.com`). A name, however, is not inverse-functional, as multiple persons can share the same name. The text notes that reasoning with this type of knowledge is discussed later.

An advantage of FOAF for data sharing is the **relative stability of its ontology**. The large number of FOAF users compels maintainers to keep the vocabulary and its semantics stable. Even inconsistencies in naming (e.g., `mbox_sha1sum` vs. Java convention) are often left uncorrected. When term meanings change, the evolution is towards **generalization** to avoid breaking existing applications. For instance, sub-properties like `foaf:knowsWell` and `foaf:friend` were removed

due to overlapping meaning, and `foaf:schoolHomepage` was broadened to include university homepages.

To facilitate adoption, new terms are generally *not* added to the vocabulary; instead, authors are encouraged to create **extensions** using RDF mechanisms like subclasses, subproperties, or new properties for existing classes. FOAF terms, especially `foaf:Person`, are frequently referenced in other ontologies. Examples include:

- **SIOC (Semantically Enabled Online Communities):** Connects discussions across various fora (Usenet, blogs). A `sio:User` (user account) is related to `foaf:Person` via `sio:account_of` property, as a person can have multiple accounts (not a subclass).
- **DOAP (Description of a Project):** For open source software projects.
- **BuRST format:** For describing publications in openacademia. In both DOAP and BuRST, FOAF vocabulary terms are adopted to describe project maintainers and publication authors.

While FOAF provides a rich ontology for individuals (especially their online presence), it is **rather poor as a vocabulary for describing relationships**. It defines only a single `foaf:knows` relationship between Persons, with no ontological restrictions and a broad definition. This broadness was intentional for wider applicability, but it has diluted the meaning of "knows" through usage. The creators expected others to use **RDF/OWL extensibility** to define more precise relationship notions. For example, one could define `example:supervises` as a `subPropertyOf foaf:knows` between `example:Teacher` and `example:Student` (subclasses of `foaf:Person`), and OWL could add constraints (e.g., every Student is supervised by at least one Teacher). However, to characterize relationships beyond just two participants, relationships themselves need to be objects of the ontology.

5.3 Ontological representation of social relationships

Extending ontological representations like FOAF with a framework for modeling and characterizing social relationships is needed for two main reasons:

1. To **support automated semantic integration** of social information.
2. To **capture established concepts** in Social Network Analysis (SNA).

This task uses an **engineering method of decomposition**, aiming for a fine-grained representation to capture details from individual sources, allowing later recombination and evidence for relationships. Network analysis itself provides a rich vocabulary for characterizing social relationships, which are prime

candidates for inclusion in an ontology. The text focuses on **interpersonal relations**, ignoring relationships at other levels (e.g., institutional).

Commonly discussed characteristics of social relationships include:

- **Sign (Valence):** Whether a relationship represents positive (like) or negative (hate) attitudes. The positive or negative charge is important for studying balance within social networks, a core concept in **balance theory**.
- **Strength:** First introduced by Granovetter's work on weak ties. Tie strength is a complex construct with many aspects, including: Affect, Frequency/frequent contact, Reciprocity, Trust, Complementarity, Accommodation, Indebtedness, Collaboration, Transaction investments, Strong history, Fungible skills, Expectations, Social capital, Bounded solidarity, Lower opportunistic behavior, Density, Maximize relationship over org., Fine-grained information transfer, Problem solving, Duration, Multiplexity, Diffusion, Facilitation, Personal involvement, Low formality (few contracts), Connectedness. The conceptualization of tie strength is "fuzzy," with little agreement on the importance of individual aspects. Researchers often ignore irrelevant aspects. A major issue is the lack of agreed-upon measurement methods, leading to incomparable data across studies. Despite this, there's a need to represent measured tie strength for purposes like secondary analysis.
- **Provenance:** A social relationship can be viewed differently by participants, potentially leading to **unreciprocated ties** (perceived by only one member). Outsiders may also provide different accounts, a known bias in SNA.
- **Relationship history:** Relationships begin with an event involving two individuals (not necessarily personal contact, but social interaction). They then have a lifecycle where characteristics can change through interaction or lack thereof.
- **Relationship roles:** Social relationships can have associated social roles (e.g., professor, student in a student/professor relationship). Both the relationship and roles may be context-dependent.

Web-based social networking services capture relationship types and attributes in various ways. For example, Orkut uses a 5-point scale for friendship strength, while LinkedIn focuses on professional exchanges. Ideally, all users would agree on a single typology, but this is neither feasible nor necessary. The requirement for such a representation is that it be **minimal to facilitate adoption** and **preserve key identifying characteristics** (like identifying properties for social individuals). For instance, two FOAF-aware systems can

determine if two person objects are the same individual even if they use different subtypes of `foaf:Person`.

In summary, a rich ontological characterization of social relationships is needed for aggregating social network information from multiple, possibly different, sources and contexts (typical on the Web and in social sciences). The proposed representation views relationships as **higher-order concepts (patterns)** that capture a set of constraints on individual interaction and can be mapped to or extracted from observations about the environment.

5.3.1 Conceptual model

The importance of social relationships suggests they should be treated as **first-class citizens** (objects, not just relations between objects). While *n*-ary relations are not directly supported by RDF/OWL, solutions exist.

The common technique for handling *n*-ary relations is **reification**. This involves representing the relation *as a class*, whose instances are concrete relations of that type. RDF itself uses reification for statements: `rdf:Statement` represents the class of statements, with properties `rdf:subject`, `rdf:predicate`, and `rdf:object` linking the statement instance to the resources involved.

To enrich FOAF relationship representation, the **reification feature of RDF language** can be used. The text proposes two alternative RDF(S) representations, both reifying the original triple asserting the relationship (e.g., a `foaf:knows` statement). Relationships become subclasses of `rdf:Statement` (Figures 5.3 and 5.4, refer to source for visual examples). In both representations, a new `Relationship` class is linked to a generic `Parameter` class by the `hasParameter` relationship. Relationship types (e.g., `Friendship`) are subclasses of `Relationship`, and their parameters (e.g., `strength`, `frequency`) are subtypes of `Parameter`. The `hasParameter` metaproperty cannot be defined in OWL DL.

The two alternatives differ in parameter representation:

1. **First scheme (OWL-S inspired):** Parameters are related to their range (e.g., `owl:Thing` or a datatype) by the `valued-by` metaproperty. For example, `Strength` could be a subclass of `Parameter` `valued-by` integers. The disadvantage is that specifying values requires two statements or a constructed property, which is not expressible in OWL.
2. **Second alternative (native RDF method):** The generic `Parameter` class is defined as a subclass of `rdf:Property`. This makes representing parameter values and restrictions more natural. However, this solution is **not compliant with OWL DL** because OWL DL does not allow

declaring properties ranging on properties or creating subclasses of `rdf:Property`.

Describing relations using statement reification has the advantage of being **directly supported by RDF** and can be efficiently stored and queried in quad-based triple stores.

However, there is **caution required when using RDF reification**:

- **No formal semantics in RDF or OWL:** The existence of a reified statement does not imply the existence of the "ground" (original) statement, or vice versa.
- **Not a form of quotation:** Many might expect reification to represent a "stating" of a particular statement attributable to someone, but this is not the case due to interaction with OWL semantics.
- **Identical instances not implied:** Two `rdf:Statement` instances with the same subject, predicate, and object are not necessarily identical.

Simple reification-based representation is also **not powerful enough to capture social relationships with arity higher than two**. For example, "brokering" (a relationship between two people *and* a broker) cannot be directly represented in RDF and thus cannot be reified.

More advanced representation needs to capture two characteristics of social relations: **context dependence** and the **separation between an observable situation and its interpretation** as a social relation. Social relations are **socially constructed objects**; their definition and identification are dependent on the **social context** in which they are interpretable. This process, called **cognitive structuring**, applies a generic pattern to observed states-of-affairs. The same observation can be interpreted as different relationships based on different patterns (e.g., a student/professor relationship is specific to a university context). The individual relationships and their generic descriptions are distinct. The generic pattern includes only shared aspects, allowing variation in particular relations.

The representation of context and the separation of "state-of-affairs" (observations of objects, events) from "descriptions" (contexts) is a common problem in human knowledge representation. The **Descriptions and Situations (D&S) ontology design pattern** [GM03] provides a model of context and clearly delineates these two layers. D&S is a generic pattern for modeling non-physical objects (like norms, plans, social roles) whose meaning emerges from statements in combination with other entities. It accounts for such objects by modeling the context or frame of reference they depend on. D&S has been successfully applied in various ontology engineering projects.

D&S builds on categories from the **DOLCE foundational ontology**: Objects, Events, and Regions. In D&S, Context is composed of Parameters, Functional Roles, and Courses of Events. Axioms ensure descriptive categories select basic DOLCE categories: Parameters are valued-by Regions, Functional Roles are played-by Objects, and Courses of Events sequence Events. Context elements mirror state-of-affairs but add semantics. This separation allows multiple overlapping or alternative contexts to match the same world, with systematic relations among their elements.

D&S has been used for social contexts and roles, and these arguments apply equally to social relations. Therefore, a Social Relationship is modeled as a subclass of Context, and specific relationships like Friendship are subclasses of Social Relationship. As contexts, Social Relationships can have Parameters, Roles, and a single Course as components.

A typical Role is the Relationship Role, a subclass of Social Role. Examples include Friend in friendship, Student/Professor in a student/advisor relationship, or Uncle/Nephew in kinship. Relationship Roles are restricted to be played by Natural Persons. The course of the relationship captures generic characteristics of events and their sequences that define a relationship. It is related to actual events in a particular relationship by the sequences relationship. Characteristics like frequency are conceptualized as parameters, often a requisite for the course of the relationship (e.g., frequency as average number of events over time). Softer qualities (emotional content) may be harder to capture precisely, but engineers should strive to relate them to other components or at least document elicitation questions.

Figure 5.6 (refer to source for visual example) illustrates the representation of a friendship relation. Friendship generally has a single role, Friend, played by actual persons (e.g., Jack and Jill). It has a typical course; an event like a dinner with both participants might be related to this course, indicating its significance to the friendship's development. (Friendship's typical course is difficult to capture precisely, though typical "beginning" events can be discerned).

This advanced representation of social relationships as higher-order concepts (patterns) mapped to concrete observations aligns well with the increasing availability of fine-grained electronic data (e.g., emails) about social interactions. This trend is expected to continue with ubiquitous devices collecting data on interactions. Technology for tracking relationships based on sensor data is accepted when participants agree and it serves a well-defined purpose, such as a ubiquitous system for supporting conference participant networking (Matsuo et al.), which uses infrared sensors and RFID cards to observe physical presence and explicit actions.

5.4 Aggregating and reasoning with social network data

The first step in aggregating data from traditional formats (databases, Excel, XML) is to **convert them into an RDF-based syntax** for storage in an ontology store and manipulation with ontology-based tools. This involves assigning **identifiers (URIs)** to resources and re-representing data using a shared ontology like FOAF.

When converting data from external sources, it's often natural to **preserve their original schema** (e.g., database table definitions). **Ontology mapping** can then unify data at the schema level by mapping classes and properties from different schemas to a shared ontology (e.g., FOAF), effectively treating the data as if it had a single shared schema. Automated ontology mapping is an active research area in the Semantic Web community, though it's not always necessary for typical cases with limited ontologies.

The aggregation task isn't complete after schema mapping; it also requires **finding identical resources across datasets**. This is a two-step process:

1. **Capturing domain-specific knowledge** about when two instances are considered the same. FOAF itself provides ways to infer equality (e.g., based on email), but domain-specific criteria based on domain-specific properties are often needed. This requires understanding the general meaning of equality in RDF/OWL. Often, equality determination involves applying a threshold to a similarity measure (a weighted combination of property similarities), which requires a **procedural representation of knowledge** as it involves computation.
2. **Instance unification, or "smushing"**: Once equality rules/procedures are determined, the actual unification is carried out. Unlike much related work, smushing is considered a **reasoning task**, where rules/procedures are iteratively executed until no more equivalent instances are found. The advantage of this iterative approach (compared to a one-step similarity computation) is that learned equalities can be incorporated into subsequent reasoning rounds.

The text will discuss the advantages and disadvantages of **rule-based reasoners** vs. **Description Logic (DL) reasoners** for this task, the trade-off between **forward- and backward-chaining reasoning**, and how to combine procedural and rule-based reasoning.

5.4.1 Representing identity

One major advantage of RDF over other formalisms like UML is the ability to **uniquely identify resources** (instances, classes, properties). The primary

mechanism for this is assigning **URIs** to resources; every resource, except blank nodes, is identified by a URI.

In practice, there are often multiple candidate identifiers for a single object. For example, publications may have ISBNs, ISSNs, DOIs, and URLs. These identifiers are usually *unique* (an identifier identifies a single resource) but mostly not *single* (a resource may be identified by multiple identifiers).

Multiple identifiers can be represented in RDF in two ways:

1. Introduce a separate resource and use the identifiers as URIs for these resources. Then, the equality of these separate resources can be expressed using the `owl:sameAs` property.
2. Choose one of the identifiers and use it as the primary URI.

Resource identifiers must conform to URI specifications and good practice. Modern schemes like DOIs are designed for this. Other schemes can be recoded as URIs with the `info:` prefix. It's common practice to create URIs within a web domain owned or controlled by the metadata creator (e.g., `http://www.example.org/id/123` for a resource with ID 123 at `example.org`). This satisfies guidelines for **good URIs**: they should be **unique** (unambiguous, unlikely to be reused for different things, preventing URI clashes) and **stable** (cannot be easily renamed, as changing a URI means introducing a new resource and asserting equivalence, which is problematic in large-scale systems). Encoding unstable property values (e.g., journal, volume, page number in a publication ID) in URIs is a bad idea because data entry mistakes would render the identifier obsolete or ambiguous.

The use of `http://` URIs has been complicated by the debate over whether they can be used for abstract concepts (like a person) or only web resources (like an HTML page). The resolution is that the web server's response code should indicate whether the URI denotes a web resource or an abstract concept. A practical consequence is that for abstract concepts, one should avoid URIs that resolve to an existing HTML page, to prevent URI clashes.

5.4.2 On the notion of equality

RDF and OWL (starting with OWL Lite) allow identifying resources and representing their (in)equality using the `owl:sameAs` and `owl:differentFrom` properties. These are basic building blocks; the *meaning* of equality depends on the domain and modeling goals. For example, what constitutes "the same" for two persons or publications is domain knowledge. Different levels of modeling might distinguish individual persons versus groups (where individuals within the group are considered equivalent, e.g., role equivalence).

The text briefly detours into philosophy to discuss the notion of identity and equality. The most well-known formulation is **Leibniz's "Identity of Indiscernibles" (Leibniz-law)**: "no two distinct objects have exactly the same properties". Loosely, all things are identical unless they can be distinguished. This is formalized as $\forall P : P(x) \leftrightarrow P(y) \rightarrow x = y$ (Formula 5.1). The converse, **Indiscernibility of Identicals**, states $\forall P : P(x) \leftrightarrow P(y) \leftarrow x = y$ (Formula 5.2). Together, these are often called the Leibniz-law and form the basis of equality definitions in many systems.

The reflexive, symmetric, and transitive properties of equality follow from these definitions. The quantification over properties makes these formulas "second-degree" and leads to different interpretations in **open and closed worlds**.

- In an **open world**, the number of properties is unknown, so the Leibniz-law is impractical; one can never conclude equality because there might always be an undiscovered distinguishing property.
- In a **closed world**, one might iterate over all properties to check equality; if no distinguishing property is found, equality is assumed. However, a closed-world assumption can be undesirable; for example, knowing only that two resources are the same gender doesn't mean they're identical.

Philosophical objections to the Leibniz-law's original form argue it's too strong, as perfectly symmetrical objects might be distinguishable by "impure, extrinsic properties" (e.g., distance to another object). The solution is to **limit the types of properties considered**, excluding such extrinsic or transient properties (e.g., `foaf:based_near` or `foaf:gender`).

Regarding **OWL semantics** and `owl:sameAs`:

- OWL's semantics are built on an **open world assumption**, so the Leibniz-law *cannot* be used to infer identity based on indiscernibility. However, equality can still be inferred by necessity.
- The semantics of `owl:sameAs` conform to Formula 5.2 (Indiscernibility of Identicals). This means `owl:sameAs` restricts the interpretation to models where the two symbols denote the *same object*, making them interchangeable in statements. This implies:
 - If `s1` is `sameAs` `s2`, and `s1` has property `p` with object `o`, then `s2` must also have property `p` with object `o`.
 - Similar rules apply for properties and objects.
- The `owl:sameAs` property is also **reflexive, symmetric, and transitive**.
- It is **not inconsistent** for resources declared `owl:sameAs` to have different *stated* properties (e.g., `s1` has name "John", `s2` has name "Paul", and `s1 owl:sameAs s2`). This is due to the open-world assumption, where

missing statements are assumed to exist elsewhere. In a closed world, this would be inconsistent.

5.4.3 Determining equality

In OWL, certain constructs can lead to equality or inequality conclusions:

- **Functional and Inverse Functional Properties (IFPs):** If a property is functional (value uniquely determines subject) or inverse functional (value uniquely determines object), and two subjects/objects share the same value for that property, they must be the same resource. For example, `foaf:mbox` is IFP, so if two individuals have the same `mbox`, they are the same person.
- **Maximum cardinality restrictions:** If a property has a maximum cardinality, and more instances are asserted than allowed, then some of those instances must be the same. For instance, if `ex:hasParent` has a max cardinality of two, and a person is stated to have three parents, an OWL reasoner would conclude that at least two of those parents must be the same.
- **Disjoint classes:** Instances of disjoint classes must be different from each other, leading to `owl:differentFrom` conclusions. Once inferred, equality can be explicitly stated using `owl:sameAs`. (Note: `owl:sameAs` can equate classes and properties in OWL Full, but not OWL DL).

Often, the knowledge needed to determine equality cannot be expressed solely in OWL, requiring **more expressive rule languages**. A typical example is **complex keys**, such as equating two persons only if both their names and birth dates match; this is not possible in OWL but can be captured in Horn logic (a common target for Semantic Web rule languages).

Even rule languages may not be sufficient in some situations. Elementary reasoning steps involving **simple data type manipulation** (e.g., string concatenation) are not part of DL or rule languages. An example is matching person names: this involves separating first and last names (a complex natural language problem), case-insensitive comparison of last names, and fuzzy string matching for first names to handle abbreviations. This algorithm is easily described in a programming language or transformation languages like XSLT, which have necessary datatypes and control structures. Rule languages in the Semantic Web context generally lack this expressive power. The solution is to **combine declarative (logic-based) and procedural reasoning**, where certain relationships are computed by external components or services that conform to a particular interface. Reasoning then becomes a combination of calling these services and executing a regular DL or rule-based reasoner.

5.4.4 Reasoning with instance equality

Reasoning is the inference of new statements (facts) that necessarily follow from known statements, determined by the language's semantics. Every inferred fact adds meaning by representing a constraint on the mapping between symbols and the domain of interpretation, excluding unintended interpretations. While an infinite number of statements could be inferred from a non-trivial knowledge base, in practice, only statements relevant to a specific task are desired, to "complete" the knowledge base.

Implementing reasoning involves trade-offs, mostly concerning **efficiency and scalability**.

Description Logic versus rule-based reasoners:

- The task here is **instance equality reasoning** (inferring `owl:sameAs` and `owl:differentFrom`).
- **OWL (DL) reasoners** are generally of limited use for this kind of reasoning. The representation needs may differ slightly from OWL (e.g., using OWL Tiny constructs, and knowledge expressible only by rules).
- While OWL reasoners can handle the OWL-expressible part, practical applications often don't benefit from the theoretical efficiency results (complexity) of OWL DL, which were designed for decidability and efficient reasoning in worst-case scenarios. In practice, average case matters, and approximate algorithms might be faster.
- DL reasoners primarily support **classification** (determining subclass hierarchies) and **consistency checking** of ontologies. Other tasks are reformulated as consistency checks. For equality, checking if two instances are the same means adding their opposite (they are different) and checking for inconsistency. If inconsistent, they are the same. This is very **inefficient** as it requires a consistency check for every pair of instances.
- A better alternative is **rule-based reasoning**, which the author explored in combination with procedural attachments. The semantics of RDF(S) and a significant part of OWL (including relevant parts like `owl:sameAs` semantics) can be expressed using inference rules. These rules can be implemented by reasoners like OWLIM or custom rule-based reasoners.

Forward versus backward chaining:

- **Forward chaining:** Computes all consequences of rules to create a **complete materialization** of the knowledge base. This involves repeatedly checking rule prerequisites and adding conclusions until no new statements are inferred.

- **Advantage:** Queries are fast because all true statements are readily available (reasoning done before query time).
 - **Disadvantage:** Storing the complete materialization often requires **exorbitant space** (typically several times the original repository size), with many inferred statements being irrelevant to the task. Updates are also complex, as removing a statement requires removing all statements inferred *only* from it.
- **Backward chaining:** Executes rules "backwards" and **on demand** (when a query needs answering). Given a conclusion, it checks if it's explicitly stated or can be inferred from other rules.
 - **Drawback: Longer query execution times.**
- **Advantage of rule-based axiomatization:** The expressivity of reasoning can be **fine-tuned by removing irrelevant rules**. For instance, the rule stating all subjects are `rdfs:Resource` can be ignored if not needed, saving space in forward chaining.
- **Flink system's approach:** Combines forward and backward chaining for efficiency and scalability. Rules for inferring instance equality are executed **iteratively in a forward-chaining manner** (querying premises, adding consequents). The semantics of `owl:sameAs` statements are partly inferred **in a backward-chaining manner**. Specifically, the reflexive, symmetric, and transitive properties of `owl:sameAs` (Formula 5.4) are handled by forward chaining, while the interchangeability in statements (Formula 5.3) is dealt with at query time by **rewriting queries**. For example, a query for Peter's email is rewritten to also return emails of instances `owl:sameAs Peter`, taking advantage of `owl:sameAs` reflexivity. This query rewriting is implemented in the Elmo API.

The timing of reasoning and the method of representation: There are three basic variations for when identity reasoning (smushing) is performed, and a choice between representing equivalence using `owl:sameAs` or directly merging descriptions:

1. **During data addition:** Smushing is carried out as data is added. Example: Ingenta's large-scale publication repository checks for equivalent instances when a new one is added.
 - **Method:** Descriptions are typically **merged**, keeping only one identifier and consolidating all information under it.
 - **Advantage:** Reduced repository size (no redundant `owl:sameAs` relations).
 - **Serious potential drawback:** If merged instances are later found to be *not* the same (due to error, changed criteria, or changed property values), it's **impossible to un-merge** the descriptions.

Using `owl:sameAs` relationships makes revising equality reasoning results very easy by simply removing statements.

2. **After the repository is filled:** This is the choice in the Flink system.
 - **Method:** A choice exists between merging identifiers or using `owl:sameAs` relationships. The benefits of merging are less obvious here because removing statements is expensive in most triple stores.
3. **At query time:** This is often the only choice, especially when querying multiple repositories and dynamically aggregating data in an interface (e.g., openacademia).
 - **Method:** Query each repository for matching instances, perform duplicate detection on the combined set, and present only the purged list to the user.

5.4.5 Evaluating smushing

Smushing (instance unification) can be considered either a **retrieval problem** or a **clustering problem**.

- As a retrieval problem: The goal is to achieve maximum precision and recall for the theoretically correct set of mappings. Unlike ontology mapping (where mappings are typically one-to-one), instance unification allows a single resource to map to multiple others.
- As a clustering problem: The task is conceptualized as clustering, and evaluation compares the generated clusters against an ideal clustering.

Instance unification can be seen as an **optimization task** aimed at optimizing an information retrieval or clustering measure. It is considered "easier" than ontology mapping because it doesn't enforce one-to-one mappings, which can limit future choices and lead to local minima. In smushing, the main concern is that if both positive rules (entailing `owl:sameAs`) and negative rules (entailing `owl:differentFrom`) are used, an **inconsistent state** (where two resources are both the same and different) could arise, indicating inconsistency in the rules or input data. The choice of evaluation measure depends on the application's costs for different types of mistakes (false positives, false negatives).

5.5 Discussion (Modelling and Aggregating Social Network Data)

Ontology languages offer a more flexible, distributed way to represent knowledge, which is particularly appealing when combining knowledge sources under independent control, as is typical on the Web or even for integrating data on a single computer.

The chapter showed how ontology-based Knowledge Representation models social individuals and relationships. Unlike traditional mechanisms, ontology languages directly support **data integration at both schema and instance levels** by providing constructs for mappings. Schema-level integration is **ontology mapping**, a vibrant research area in Semantic Web. The primary concern, however, was **instance-level integration (smushing)**. The OWL ontology language supports this with rigorous identification, declarative means to describe equality/difference (e.g., cardinality restrictions, disjointness), and the `owl:sameAs` and `owl:differentFrom` properties.

However, a significant portion of the desired knowledge could not be expressed in OWL.

5.5.1 Advanced representations

Some knowledge could not be captured in OWL DL. Missing features that don't significantly change complexity (like `owl:ReflexiveProperty`) are likely to appear in future OWL revisions. However, OWL will remain constrained by original complexity concerns, which often don't apply to practical scenarios. More expressive power is expected from the **Rule Interchange Format (RIF)**, likely including first-order logic (FOL).

Logic can be extended further:

- **Temporal logic:** Extends assertions with a **temporal dimension** (statement holds at a certain time or interval). It's required for formalizing **ontology versioning** (change management). For information integration, changes can affect instance properties or class descriptions (including equivalence rules). While conceptualization should be stable, sometimes matching on dynamic properties (like hair color) is desired, requiring consideration of the time assertions were true.
- **Probabilistic logic:** A natural step for more accurate problem representation, as property matching often yields **probabilistic statements** (e.g., string matching algorithms providing similarity measures). These can be combined with weights (determined ad hoc or by machine learning) to form a probability that two persons are the same. This is distinct from fuzzy logic; here, instances are either the same or not, but with uncertainty. The approach described applies a **threshold to such probabilities to return to binary logic**.

6 Developing Social-Semantic Applications

This chapter contrasts Semantic Web applications with traditional web applications (e.g., LAMP stack - Linux, Apache, MySQL, PHP). The key differences are:

- The **kind of data dealt with**: Ontologies with formal semantics.
- The emphasis on the **distributed nature of applications**.

The characteristics defining a Semantic Web application, as per the Semantic Web Challenge, are:

1. **Meaning of data plays a central role**: Meaning must be formally described, data manipulated to derive useful information, and this semantic processing must be central to achieving goals that alternative technologies cannot.
2. **Information sources**: Should be under **diverse ownership/control**, **heterogeneous** (syntactically, structurally, semantically), and contain **substantial real-world data** (not just toy examples).
3. **Open world assumption**: All applications must assume that information is never complete.

The novelty of Semantic Web development lies in building applications on **distributed, heterogeneous information sources**. Exchanging data across domains with different ownership and interests necessitates new formats capable of capturing data meaning. Traditional web applications (e.g., online bookstores) typically rely on a single, controlled data source (relational database), where data meaning is known and consistency is assured, allowing reasoning to be hard-coded. Semantic Web applications, by contrast, deal with distributed data of varying quality, formatted with different but potentially overlapping schemas, and containing overlapping instances.

Developers new to Semantic Web face a **steep learning curve**, needing to familiarize themselves with ontology-based KR technologies (RDF/OWL, RDF/XML, SPARQL) and new tools for semantic data processing. This has hindered adoption, with many developers dissatisfied when applying "heavy-weight" technologies to trivial problems, where performance costs outweigh immediate benefits. The advantages often manifest in the long run when applications need to combine with external data or repurpose data. There's also a "chicken and egg" problem: lack of applications due to lack of RDF data, and vice versa.

Some problems have been resolved: **scalability of ontology databases (triple stores)** has improved, reducing performance loss when using RDF as a native data format. Developers are more motivated as the number of available ontologies and data sources increases. However, the **development overhead**

remained a key issue. To lower this threshold, the author developed a set of programming tools, primarily the **Elmo toolkit**.

Elmo enables software engineers to manipulate RDF/OWL data at the **domain concept level** (e.g., persons, publications), abstracting away the lower-level ontology language (resources, literals, statements). It acts as a middle layer between the RDF ontology store and the application, hiding much complexity and allowing developers to focus on application logic. Elmo also includes tools for RDF data, FOAF data, instance unification, and a validation framework for incorrect data. Additionally, **GraphUtil** maps FOAF network data to the graph model of the JUNG (Java Universal Network Graph) API, allowing network analysis concepts (vertices, edges) and statistics (centrality measures).

The text then describes the general design of Semantic Web applications, then introduces Sesame, Elmo, and GraphUtil, using Flink and openacademia as examples.

6.1 Building Semantic Web applications with social network features

Most current Semantic Web applications share a generic design. They are essentially **mashups**, building on heterogeneous data sources and services under diverse ownership.

Before external, heterogeneous data sources can be reused, they require **normalization**:

- **Syntactic normalization**: Transforming data into an RDF syntax like RDF/XML. This step can be skipped if data is already exposed as RDF/OWL or queryable via SPARQL.
- **Semantic normalization**: Reconciling the ontologies (schemas and instances) of data sources. For fixed, small numbers of data sources, schemas can be known in advance and mapped manually. Future applications are expected to discover and map new data sources automatically.

Semantic Web applications **persist information in ontology stores (triple stores)**, databases designed for storing, manipulating, and querying RDF/OWL data. Triple stores almost always have or can connect to a **reasoner**, used to infer new information or check consistency. Some allow custom rules. Instance unification can also be partly solved by OWL DL or rule-based reasoning. Reasoning can occur during data addition (forward-chaining) or at query time (backward-chaining).

Most Semantic Web applications have a web interface, but this isn't a requirement. They are expected to expose data using standard Semantic Web languages and protocols (e.g., **SPARQL**, **REST-style interfaces**). A SPARQL service allows querying but no data manipulation.

Application logic resides between the triple store and the web interface. The application typically accesses the triple store via its **client API**, manipulating data at the level of RDF triples (adding/removing triples, queries returning tables of variable bindings). This is similar to relational database access. A key issue is the lack of standard protocols like ODBC/JDBC for relational databases, meaning application code is often specific to a particular triple store without additional abstraction layers.

It's often desirable to access a triple store at an **ontological level** (classes, instances, properties), which aligns with object-oriented frameworks. The **Elmo library** facilitates this by mapping ontological data in the triple store to Java classes, where setting/reading attributes translates to adding/removing corresponding triples. Elmo is implemented for Sesame but can be adapted for other Java-based triple stores if standard protocols like JDBC become available for non-Java stores.

6.1.2 Sesame

Sesame is a **Java-based triple store**. Like a database for RDF, it allows creating repositories, specifying access privileges, storing RDF data, and querying using supported languages (Sesame's **SeRQL** and **SPARQL**). SeRQL is more expressive than SPARQL. Data is manipulated at the triple level (add/remove statements). Sesame can persistently store data in memory, on disk, or in a relational database.

Sesame integrates **reasoning**. It has a built-in inferencer for RDF(S) inference rules. While it doesn't support full OWL semantics, its rule language can capture most of OWL's semantics, including **inverse-functional properties** and the semantics of the `owl:sameAs` **relationship**. Reasoning can be enabled/disabled; when enabled, it's performed during data addition or removal.

A recently added feature is the ability to store and retrieve **context information** (also known as **named graphs**). This is useful in distributed scenarios for metadata about statements (e.g., provenance, timestamp for FOAF profiles) or for centralized user-contributed content. Earlier versions required inefficient RDF reification for context. Since Sesame 2.0, the repository natively supports context storage and querying, effectively making every triple a **quad** (subject, predicate, object, context ID). Contexts are identified by resources and can be queried directly using SPARQL.

Sesame functionalities are accessible in three ways:

1. **HTML interface** via a browser.
2. **Servlets** exposing remote access via HTTP, SOAP, and RMI.
3. **Java client library**: Provides access to all functionality via method calls on a `SesameRepository` object, connecting to local or remote Sesame servers.

Working with the Sesame client API is straightforward; queries are executed using `evaluateTableQuery`, returning a `QueryResultsTable`. Values in the table are `URI`, `BNode`, or `Literal` objects, with methods to access their string representation, datatype, and language. However, Sesame's client library operates at the level of **individual triples**. Object-oriented applications manipulate data at the level of objects and their attributes, where a single attribute update might translate to multiple triple updates, and removing an object means removing many triples. This highlights the need for an API that translates between object operations and the underlying triple representation, which is a main concern for the Elmo API.

6.1.3 Elmo

Elmo is a development toolkit with two main components:

1. **Elmo API**: Provides the interface between JavaBeans representing ontological classes and the underlying triple store. It includes tools for generating JavaBeans from ontologies and vice versa.
2. **Set of tools for working with RDF data**: Includes an RDF crawler and a framework of smushers (instance unification methods).

The Elmo API: The core is the `ElmoManager`, a JavaBean pool implementation responsible for creating, loading, renaming, and removing **ElmoBeans**.

`ElmoBeans` are compositions of **concepts** and **behaviors**.

- **Concepts**: Java interfaces that correspond one-to-one to ontological classes, providing getter and setter methods for ontological properties. The ontological inheritance hierarchy maps directly to the concept inheritance hierarchy. Elmo concepts are typically code-generated.
- **Behaviors**: Concrete or abstract classes used to provide specific implementations for concept methods (if differing from default) or add additional functionality. Behaviors can be "mixed-in" to `ElmoBeans` dynamically. Instances of `ElmoBeans` correspond to instances in the dataset. Since an ontology resource can have multiple types, `ElmoBeans` can be composed of multiple concept interfaces. Separate classes may need to be generated for every combination of types occurring in the data,

as Java instances cannot belong to multiple classes. `ElmoBeans` can be generated at runtime as resource types change. The separation of concepts and behaviors, and their dynamic composition, supports distributed application development common in web applications.

Elmo provides pre-packaged `ElmoBean` representations for popular Web ontologies like FOAF, RSS 1.0, and Dublin Core. For instance, a `Person` `JavaBean` in the FOAF model has `foaf:Person` properties, and manipulating these properties manipulates the underlying RDF data. This higher-level representation greatly simplifies development; a simple FOAF profile can be created in ten lines of Java code (Figure 6.4, refer to source for example code). All interaction with the repository content is encapsulated by `ElmoManager`.

An `AugurRepository` module improves performance through **(predictive) caching**. Information read from the repository is cached, and writes are cached until transaction commit. Caching also involves predicting queries and pre-loading information. When a resource is first accessed, all its properties are preloaded. Another strategy tracks queries from which resources were retrieved; if a property is later read from such a resource, the same property is retrieved for all resources originating from that same query. Optionally, Elmo can perform the **query rewriting** introduced in Section 5.4.4; for example, a `getName` call on a `Person` instance can return names from all `owl:sameAs` instances.

Elmo also helps developers build applications robust against **incorrect data**, a common issue on the Web where external data has few correctness guarantees. Many RDF documents are syntactically incorrect, semantically inconsistent, or violate vocabulary assumptions, often due to human error (e.g., using a `Literal` for `foaf:mbox` instead of a URI with `mailto:` protocol).

- **Syntax validation:** Can be checked by validators (e.g., W3C's RDF validation service).
- **Inconsistency checking:** By OWL DL reasoners.
- **Programmatic checks:** Elmo provides solutions for checks only possible programmatically (e.g., `foaf:mbox` starting with `mailto:`). (Note: The `foaf:mbox` `Literal` mistake would also be caught by an OWL DL reasoner as it's an `owl:ObjectProperty`). Using **aspect-oriented programming**, **interceptors** can be added to `JavaBean` setter methods to validate incoming information. **Validators** can also check existing data. Programmers choose whether to halt processing on failure or try to recover (e.g., by removing/correcting erroneous data).

Elmo tools:

- **Elmo scutter:** A generic **RDF crawler** that follows `rdfs:seeAlso` links in RDF documents, typically pointing to other relevant RDF sources. It is used to collect FOAF profiles from the Web. Advanced features include:
 - **Blacklisting:** Automatically placing sites producing large amounts of uninteresting FOAF data (e.g., social networking services) on a blacklist.
 - **Whitelisting:** Limiting the crawler to a specific domain (URL pattern).
 - **Metadata:** Optionally stores provenance (source URL) and timestamp (collection time) for collected statements.
 - **Filtering:** Individual incoming statements can be filtered to remove unnecessary information (e.g., from unknown namespaces).
 - **Persistence:** Saves its state to disk when stopped, allowing crawling to resume from where it left off. Can preload visited URLs from the repository.
 - **Preloading from Google:** The scutter queue can be preloaded by searching Google for FOAF files.
 - **Logging:** Provides detailed crawling logs using SLF4J.
- **Elmo smusher:** Finds equivalent instances in large datasets, a common problem with FOAF profiles where the same individual may be described with different identifiers or blank nodes. Elmo provides two kinds of smushers:
 - **Class-specific comparators:** Implementations for comparing `foaf:Person` objects based on name, email, and other identifying properties, and for publications based on property combinations.
 - **Property-by-property comparison:** Compares instances based on a certain property, for example, inferring equality using inverse functional properties with a `CONSTRUCT` query. This query constructs `owl:sameAs` relationships where two resources share the same value for an IFP. The smushers report results by calling methods on registered listeners, which can output HTML or represent matches using `owl:sameAs` statements in a Sesame repository. Smushers can also act as a **wrapper**, comparing instances in a *source* repository to a *target* repository, lifting (copying) matched results. This is useful for importing information from a large general store into a specific repository.

6.1.4 GraphUtil

GraphUtil is a simple utility that facilitates reading FOAF data into the **Java Universal Network Graph (JUNG) API**. It's configured with two queries that define the nodes and edges from RDF data. For FOAF, the first query typically

returns `foaf:Person` instances (nodes), and the second returns `foaf:knows` relations (edges). Any other graph structure definable by queries can be read.

JUNG is a Java library providing an object-oriented representation of various graph types (sparse, dense, directed, undirected, k-partite). It includes implementations of well-known graph algorithms (e.g., Dijkstra's shortest path) and allows computing social network measures (e.g., centrality variations) via its `Ranker` interface. The author extended this with `PermanentNodeRanker` to store and retrieve node rankings in an RDF store. JUNG also offers a customizable visualization framework for graphs, allowing choice of layout algorithms and interaction definition (clicking, drag-and-drop). This visualization component can be used in applets.

6.2 Flink: the social networks of the Semantic Web community

Flink was the first system to leverage semantic technologies for **network analysis based on heterogeneous knowledge sources**, winning the Semantic Web Challenge in 2004. It's a general system adaptable to any community with substantial electronic data. The public instantiation of Flink presents the professional work and social connectivity of **Semantic Web researchers**. This community is defined as researchers who submitted publications or held organizing roles at past International Semantic Web Conferences (ISWC) or the Semantic Web Working Symposium of 2001. It currently covers 744 researchers globally.

Information sources include HTML pages, emails, and publications – natural byproducts of a community's daily work. Flink extracts knowledge about social networks and consolidates it using a common **semantic representation: the FOAF ontology**.

The *raison d'être* (reason for being) of Flink is threefold:

1. **Demonstration of Semantic Web technology:** Interesting for those developing systems using SW tech.
2. **Portal for the Semantic Web community:** Provides profiles, emails, publications, and statistics. Aims to **bootstrap the nascent FOAF-web** by allowing researchers to export Flink's knowledge in FOAF format as a starting point for their own profiles.
3. **Social network analysis:** The collected data is used to study the nature of power and innovativeness in scientific communities.

6.2.1 The features of Flink

Flink takes a **network perspective**, organizing website navigation around the social network of researchers. After selecting a researcher, the system displays a summary page with profile information and links to other known researchers. The immediate ego-network (neighborhood) is also visualized graphically (Figure 6.5, refer to source for visual example).

Profile information and the social network are based on analysis of web pages, emails, publications, and self-created profiles. Information includes name, email, homepage, image, affiliation, geographic location, interests, conference participation, public emails, and Semantic Web publications. External links provide access to full texts of emails and publications. At the time of writing, the system contained data on 7269 publications and 10178 mailing list messages.

Navigation can continue by clicking on co-authors, addressees, or other known individuals. A separate page shows a **summary of the relationship** between two researchers, including evidence such as link weight, physical distance, common friends, interests, and depictions, as well as shared emails and publications. Researcher interests are also used to generate a **lightweight ontology of the Semantic Web community**, where concepts are research topics and associations are based on the number of researchers interested in a pair of topics (Figure 6.6, refer to source for visual example). This ontology's associations are specific to the community used in the experiment, reflecting their conceptualizations, and evolve naturally as topic relationships change.

Visitors can view **basic social network statistics**: degree, closeness, and betweenness (measures of importance/influence), and degree distribution (a general network characteristic) (Figure 6.7, refer to source for visual example). Geographic visualizations offer an overview of researcher locations and connections (Figure 1.4, refer to source for visual example).

6.2.2 System design

Flink's architecture, like most data-driven applications, has three layers: **metadata acquisition, storage, and presentation**.

Information collection layer: This layer acquires metadata from four types of knowledge sources: HTML pages, FOAF profiles, public email collections, and bibliographic data. All collected knowledge is represented using a common ontology, including FOAF and minimal extensions.

- **Web mining:** Employs a **co-occurrence analysis technique**. It also finds **topic interests** (associating researchers with research areas). Network ties, interest associations, and other metadata are represented in RDF

using FOAF terms (e.g., `foaf:knows` for relationships, `foaf:topic_interest` for interests). A **reification-based extension of the FOAF model** is needed to represent association weights.

- **FOAF profiles:** The native format. Gathered using the **Elmo scutter**, starting from the author's profile. The scutter is focused, collecting only profiles related to community members and limited to relevant statements (predicates in RDF, RDF-S, FOAF, WGS-84 namespaces) to control data volume. **Elmo's context features** record provenance (source URL, collection time) for statements.
- **Emails:** Processed in two steps:
 1. Download emails from POP3/IMAP, capture relevant header info in RDF, using FOAF for sender/receiver names and email addresses. (A custom ontology is used for other header info as there's no common email ontology).
 2. **Smushing:** Matching `Person` instances from emails with instances describing the target community.
- **Bibliographic information:** Collected by querying Google Scholar with names (plus disambiguation terms). Extracts title, URL, publication year, and citations. Publication metadata is represented using the **Semantic Web Research Community (SWRC) ontology**, which maps BibTeX types/fields to RDF classes/properties. An alternative source (used previously) is the Bibster peer-to-peer network, which exports metadata in SWRC format.

Storage and aggregation layer: This middle layer stores and aggregates metadata. Aggregation involves **ontology mapping** and **instance reasoning**.

- **Ontology mapping:** Straightforward due to small, stable, lightweight web ontologies (SWRC and FOAF). Mappings are static and manually inserted into the knowledge base. Data not in these two ontologies is currently discarded; incorporating unknown schemas would require automated ontology mapping.
- The aggregated RDF data is stored in a **Sesame server**. Since the model extends FOAF, it's compatible with FOAF-compatible tools. A generic component finds geographical locations (lat/long) for place names in FOAF profiles by invoking the ESRI Place Finder Sample Web Service.
- **Scalability:** Sesame server offers high performance for millions of triples, especially with native repositories or in-memory storage. Upload speed is crucial for the RDF crawler, but it drops when custom rules need evaluation. Query optimization is a challenge; manual rewriting (e.g., reordering terms) and careful judgment of trade-offs between many small queries vs. single large queries (memory footprint vs. communication overhead) are needed.

- **Reasoning to enrich data:** `foaf:knows` relations are inferred between email senders/recipients and publication co-authors.
- Network statistics are **pre-computed and stored** because real-time computation is too slow. Statistics are stored separately and kept longer, showing changes over time.

User interface layer: Flink's UI is a **pure Java web application based on the Model-View-Controller (MVC) paradigm** (using Apache Struts Framework). MVC separates concerns: model (data), controller (application logic), and view (web interface).

- **Model objects:** Elmo beans representing persons, publications, emails.
- **Controller:** Retrieves necessary beans from the store and passes them to the view layer.
- **GraphUtil:** Used to read the social network from the repository, which is then handed to the visualization. The network, like Elmo beans, is kept in memory for performance.
- **View layer:** Servlets, JavaServer Pages (JSP), and Java Standard Tag Library (JSTL) generate the front-end, decoupling design from application logic.
- **Interactive visualization:** Java applets on parts of the site for interactive social network visualization. Applets communicate with a servlet to retrieve and serialize network parts, then compute layout. Users can pan and zoom. The flexibility of the interface is important due to many presentation possibilities (text-only profiles to fully graphical browsing), and because it allows tailoring presentation based on community size or other factors. Flink's unique social network presentation means it doesn't benefit from generic Semantic Web portal generators like HayStack. The UI also provides data export mechanisms: Pajek format for network analysis, RDF/XML for individual profiles, and marker files for XPlanet (geographic visualization).

6.3 openacademia: distributed, semantic-based publication management

openacademia addresses the redundant effort of researchers maintaining personal references and web pages, and creating joined publication lists for groups. It also provides a new way to instantly notify peers of new works. openacademia is a **distributed system**; a public website is available for general use, and it can be installed locally by research groups.

6.3.1 The features of openacademia

- **HTML publication list generation:** Generates an HTML representation of a personal publication collection, publishable on the Web by inserting

a single line of JavaScript into a homepage. The list is dynamically inserted, so no separate updates are needed when the collection changes. Appearance is customizable via stylesheets.

- **RSS feed generation:** An RSS feed can be generated from the collection, allowing visitors to subscribe using news readers and receive notifications when new publications are added (**information push**). Generic tools (browser extensions, online aggregators, news clients) can consume RSS feeds. Mozilla Firefox natively supports RSS feeds for dynamic bookmark folders.
- openacademia's RSS feeds are **RDF-based** and can be consumed by RDF-aware software like the Piggy Bank browser extension.
- **Group installations:** Research groups can install their own openacademia server. Members submit publications by creating a FOAF profile pointing to their collection. The system then creates a unified group publication list and RSS feed.
- **AJAX-based interface:** For browsing and searching the publication collection (Figure 6.11, refer to source for visual example). Offers various visualizations:
 - **Tagcloud:** Important keywords from titles, with size indicating importance.
 - **Co-authorship networks:** Interactive applet (like Flink).
 - **Timeline:** Interactive visualization of publications along a scrollable timeline (Figure 6.12, refer to source for visual example).
- **Keywords/tags:** Can be added manually (BibTeX, EndNote), extracted automatically from titles, or imported from blog comments via blog search engines.

6.3.2 System design

openacademia's architecture follows Flink's design: an RDF repository in the middle, populated by information sources, and queried by services (Figure 6.13, refer to source for visual example). The main difference is **dynamics**: Flink refreshes semi-automatically every few months, while openacademia repositories refresh daily automatically. For publication feeds generated from single BibTeX/EndNote files, the process is on-the-fly using an in-memory repository discarded after use.

Information collection:

- **Publication metadata:** Relies on BibTeX and EndNote formats. Authors include BibTeX files on their public websites.

- **FOAF profiles:** Authors fill a form for a basic FOAF profile (name, reference location), or reuse existing profiles. The **Elmo crawler** collects these, restricted to an institute's domain if desired.
- **BibTeX-2-RDF service:** Translates BibTeX files to RDF, creating instance data for the SWRC ontology. A simple SWRC extension (`swrc-ext:authorList`, `swrc-ext:editorList` with `rdf:Seq` range) preserves author sequence.
- **LDAP database:** At Vrije Universiteit, a service dynamically queries the local LDAP database to represent content as FOAF profiles (group membership, names, emails). Email hashes are used as identifiers for privacy. This delegates user data maintenance to existing infrastructure.

Storage and aggregation:

- The system handles **semantic heterogeneity** at both schema and instance levels, arising from distributed, web-based information.
- **Schema heterogeneity:** Handled easily as SWRC and FOAF are stable, lightweight ontologies; mappings are static and manually inserted (e.g., `swrc:Person` is `rdfs:subClassOf` `foaf:Person`).
- **Instance heterogeneity:** Arises from different identifiers for the same real-world objects in FOAF data and publication information (e.g., same author/publication in multiple BibTeX sources).
- The **Elmo smusher framework** matches `foaf:Person` instances (based on name and IFPs) and publications (exact date match, tight fuzzy title match). Matching publications based on authors is future work.
- **Instance matches** are recorded in the RDF store using the `owl:sameAs` property. Earlier versions used Sesame's custom rule language to expand `owl:sameAs` semantics (reflexive, symmetric, transitive, equality of property values), which added many statements but allowed fast queries. The current solution uses the **combined forward and backward chaining approach** outlined in Section 5.4.4.
- **Semantic technology infers additional knowledge:** A rule can state that publication co-authors know each other (`foaf:knows` relations). This enriches profiles and further disambiguates individuals based on friends.

Presentation: After merging, the triple store is queried to produce publication lists based on various criteria (personal, group, publication facets). The online interface helps users build these queries, which are processed by a web-based **publication web service**.

- **Example query (Figure 6.14, SeRQL):** Returns all publications authored by AI department members in 2005, identified by their homepage (using `foaf:homepage` as an IFP).

- This query's successful resolution relies on **schema and instance matching**. It requires publication data, department membership, and personal data from various sources (LDAP, self-maintained profiles). This highlights a key advantage of the Semantic Web: **separation of concerns**. Researchers can update profiles/publication lists independently, and LDAP maintainers can manage users as before; all information connects and merges automatically.
- Matching publications from different BibTeX sources (with varying metadata and author spellings) is crucial for correct query answering and is handled by the **publication smusher**. The `DISTINCT` keyword in SeRQL ensures unique URIs, but further post-processing might be needed for duplicate publications with different URIs due to lack of OWL equivalence support in SeRQL.
- The **publish service** takes a query, repository location, RSS channel properties, and style instructions. It queries the repository, performs post-processing, and generates an **RSS channel** with matching publications.
- The resulting channel appears as an **RSS 1.0 channel** for compatible tools while preserving RDF metadata. The method of mixing publication metadata into an RSS channel is described in the **BuRST specification**. BuRST relies on RSS parsers ignoring unknown information, so `burst:publication` property is used to attach publication metadata to RSS items (Figure 6.15, refer to source for example).
- The presentation service can add XSL stylesheet information to RSS feeds to generate different HTML layouts (tables, citation lists, detailed descriptions). This HTML output can be viewed with any XSLT-capable browser and tailored with CSS. Stylesheets also generate XML input for the Timeline widget or reproduce BibTeX/EndNote representations.

6.4 Discussion (Developing Social-Semantic Applications)

The effort required to develop Semantic Web applications has **significantly dropped** in recent years. This is attributed to:

- **W3C standardization**: Finalizing RDF, OWL (2004), and SPARQL (2006) provided interoperability.
- **Maturity of open-source tools**: Sesame storage facility, for example, has improved in robustness and scalability.
- **Programming tools like Elmo API**: Reduce the learning curve and adaptation for average Java developers. Elmo shields developers from low-level representation details by offering static object models for common web ontologies (FOAF, RSS, Dublin Core). It integrates with Sesame for services like caching.

Elmo provides common components for most Semantic Web applications. As demonstrated by Flink and openacademia, typical tasks include:

- **Crawling (scuttering) RDF data:** From web documents and structured data not originally intended for these applications (email archives, publication databases). This dynamic data collection from varied sources is characteristic of Semantic Web applications, where semantic technology offers the greatest benefits. The Elmo scutter is used by both Flink and openacademia.
- **Aggregating instance data:** From heterogeneous sources, automated by Elmo's extensible smusher framework. Instance reasoning often requires procedural components because the necessary knowledge cannot be fully represented in ontology languages, meaning reasoners alone aren't sufficient.

Flink and openacademia, despite addressing different problems (network analysis vs. publication management), **reuse many components by building on Elmo**, allowing developers to focus on specific "business logic" and minimize code complexity. Flink will be used as a tool for data collection and management in the study of the Semantic Web research community, justifying the use of electronic data by examining the relationship between real-world and online networks in a scientific setting.

Would you like to review any specific section in more detail, or perhaps test your understanding with a short quiz on the material?